

Factory Design Pattern in C# with Real-Time Example

In this article, I will discuss the **Factory Design Pattern in C#** with Examples. The Factory Design Pattern is one of the most frequently used design patterns in real-time applications. The Factory Design Pattern falls under the **Creational Design Patterns** Category. As part of this article, we will discuss the following pointers related to the Factory Design Pattern.

1. **What is the Factory Design Pattern?**
2. **Understanding the Factory Design Pattern with Real-Time Examples.**
3. **Example without using the Factory Pattern in C#.**
4. **Understanding the Problem of not using the Factory Design Pattern**
5. **Implementing the Factory Design Pattern in C#?**
6. **Understanding the UML Diagram of Factory Design Pattern.**
7. **When to Use Factory Design Pattern in Real-Time Application?**

What is Factory Design Pattern in C#?

According to Gang of Four (GoF), a **factory** is an object used to create other objects. In technical terms, a **factory** is a class with a method. That method creates and returns **different objects based on the received input parameter**.

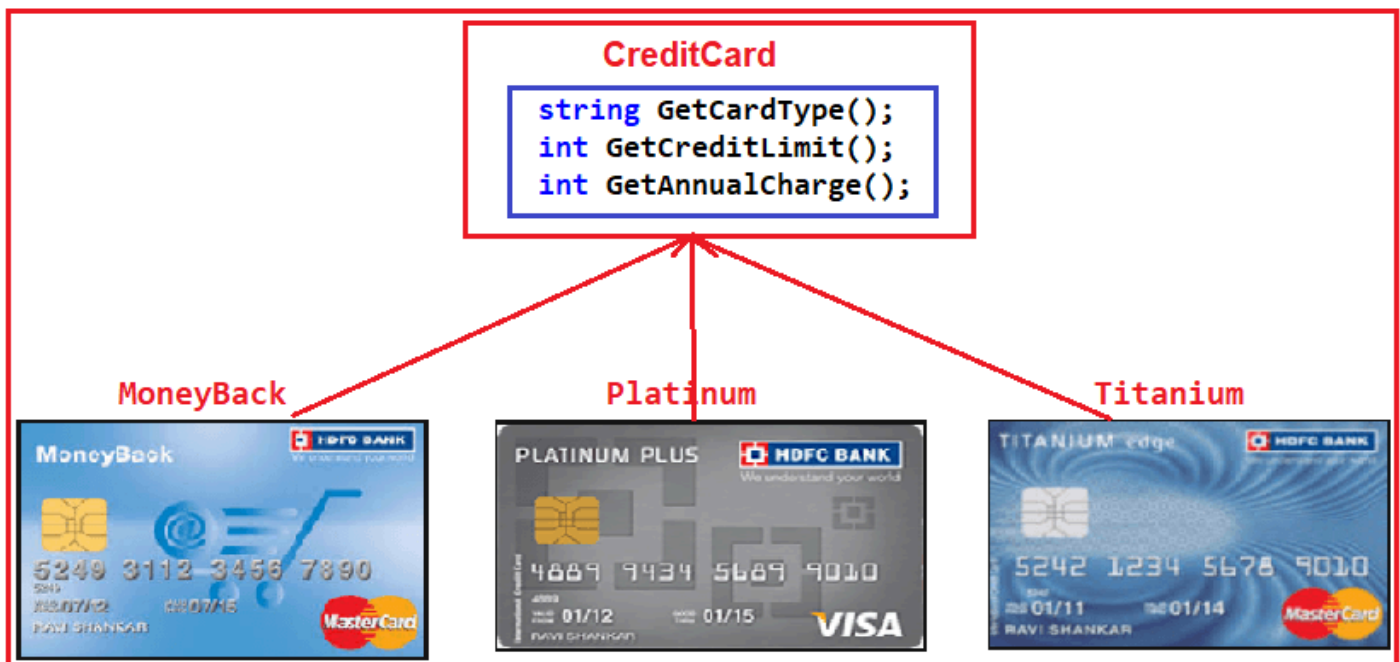
In simple words, if we have a superclass and n number of subclasses, and based on the data provided, if we have to create and return the object of one of the subclasses, then we need to use the Factory Design Pattern in C#.

In the Factory Design pattern, we create an object without exposing the Object Creation and Initialization logic to the client, and the client will refer to the newly created object using a common interface. The basic principle behind the Factory Design Pattern is that, at run time, we get an object of a similar type based on the parameter we pass. So, the client will get the appropriate object and consume the object without knowing how the object is created and initialized.

Real-Time Example to Understand Factory Design Pattern in C#

Let us understand the Factory Design Pattern with one Real-Time Example. We are going to develop an application for showing Credit Card Details.

Please have a look at the following diagram. As you can see, we have three credit card classes: MoneyBack, Titanium, and Platinum. These three classes are the subclasses of the CreditCard superclass or, you can say, super interface. The CreditCard superclass or super interface has three methods: GetCardType, GetCreditLimit, and GetAnnualCharge. The subclasses, MoneyBack, Titanium, and Platinum, have implemented the above three methods of the CreditCard.



We are required to ask the user to select the credit card. Once the user selects the credit card, we need to display the required information about that selected credit card. Let us first discuss achieving this without using the Factory Design Pattern in C#. Then, we will discuss the problems, and finally, we will create the same application using the Factory Design Pattern in C#.

Example Without using Factory Design Pattern in C#

Let's first see the example without using the Factory Design Pattern. It's very simple and straightforward. Let's proceed and implement this step by step.

Step 1: Create the Abstract Product or Product Interface (CreditCard)

Here, we need to create either an interface or an abstract class that will expose the operations a credit card should have. So, create a class file named **CreditCard.cs** and copy and paste the following code. As you can see, we created the CreditCard interface with three methods per our requirements.

```

namespace FactoryDesignPattern
{
    public interface CreditCard
    {
        string GetCardType();
        int GetCreditLimit();
        int GetAnnualCharge();
    }
}
  
```

We need to create three Product classes to implement the above interface.

Step 2: Creating Product Classes (MoneyBack, Titanium, and Platinum)

In our example, we have three Credit cards. So, we need to create three classes by implementing the CreditCard Interface and providing implementation to all three CreditCard methods. First, create a class file named **MoneyBack.cs** and copy and paste the following code.

```

namespace FactoryDesignPattern
{
  
```

```

class MoneyBack : CreditCard
{
    public string GetCardType()
    {
        return "MoneyBack";
    }
    public int GetCreditLimit()
    {
        return 15000;
    }
    public int GetAnnualCharge()
    {
        return 500;
    }
}
}

```

As you can see, this class implements the CreditCard interface and provides implementations for all three methods. We must do the same for the other two credit card classes.

Titanium.cs:

Create a class file named **Titanium.cs** and copy and paste the following code.

```

namespace FactoryDesignPattern
{
    public class Titanium : CreditCard
    {
        public string GetCardType()
        {
            return "Titanium Edge";
        }
        public int GetCreditLimit()
        {
            return 25000;
        }
        public int GetAnnualCharge()
        {
            return 1500;
        }
    }
}

```

Platinum.cs:

Create a class file named **Platinum.cs**, and copy and paste the following code.

```

namespace FactoryDesignPattern
{
    public class Platinum : CreditCard
    {
        public string GetCardType()
        {
            return "Platinum Plus";
        }
    }
}

```

```

    }
    public int GetCreditLimit()
    {
        return 35000;
    }
    public int GetAnnualCharge()
    {
        return 2000;
    }
}
}

```

So, we have created three Product classes that implement the CreditCard interface. Next, we must consume the Product classes inside the client code by creating and initializing the appropriate Product class object.

Step 3: Client Code (Main Method)

Client Code is nothing but the class from where we need to consume the product classes (MoneyBack, Titanium, and Platinum). And in our example, it will be the Main method of the Program class. We will ask the user to select the Credit Card Type in the client code. Based on the Selected Credit card, we will create an instance of any of the above three product implementation classes (MoneyBack, Titanium, and Platinum) and call the methods to show the credit card details. So, modify the Main method of the Program class as follows.

```

using System;
namespace FactoryDesignPattern
{
    class Program
    {
        static void Main(string[] args)
        {
            //Generally we will get the Card Type from UI.
            //Here we are hardcoded the card type
            string cardType = "MoneyBack";
            CreditCard cardDetails = null;
            //Based of the CreditCard Type we are creating the
            //appropriate type instance using if else condition
            if (cardType == "MoneyBack")
            {
                cardDetails = new MoneyBack();
            }
            else if (cardType == "Titanium")
            {
                cardDetails = new Titanium();
            }
            else if (cardType == "Platinum")
            {
                cardDetails = new Platinum();
            }
            if (cardDetails != null)
            {
                Console.WriteLine("CardType : " +
cardDetails.GetCardType());
                Console.WriteLine("CreditLimit : " +
cardDetails.GetCreditLimit());
            }
        }
    }
}

```

```

        Console.WriteLine("AnnualCharge :" +
cardDetails.GetAnnualCharge());
    }
    else
    {
        Console.Write("Invalid Card Type");
    }
    Console.ReadLine();
}
}
}

```

The above code implementation is very straightforward. Once we get the CardType value, we create the appropriate Credit Card instance using the IF-ELSE Condition. Then, we call the three methods to display the credit card information on the console window. So, when you run the application, you will get the output as expected, as shown below.

```

CardType : MoneyBack
CreditLimit : 15000
AnnualCharge :500

```

What is the Problem with the above Code Implementation?

The above code implementation introduces the following problems

1. First, the **Tight Coupling** between the client class (Program) and Product Classes (MoneyBack, Titanium, and Platinum). So, when we make changes in one class, we must also make changes in the other classes.
2. Secondly, suppose we add a new Credit Card. In that case, we also need to modify the client code, i.e., the main method of the Program class, by adding an extra **IF-ELSE Condition**, which not only overheads the development but also the testing process.

Let us see how to overcome the above problem by using the Factory Design Pattern in C#.

Factory Design Pattern Implementation in C#:

According to the definition of the Factory Design Pattern, the Pattern creates an object without exposing the object creation logic to the client, and the client refers to the newly created object using a common interface.

Please have a look at the following image. Our factory class is responsible for creating and returning the appropriate Product (i.e., MoneyBack, Titanium, and Platinum) object. As you can see, this class has one static method, i.e., GetCreditcard, and this method takes one input parameter and, based on the parameter value it will create one of the credit card (i.e., MoneyBack, Platinum, and Titanium) objects and store that object in the superclass (CrditCard) reference variable and finally return that superclass reference variable to the caller of this method i.e. to the client or you can say in our example it is the Main method of the Program class.

CreditCardFactory

```
public static CreditCard GetCreditCard(string cardType)
{
    CreditCard cardDetails = null;

    if (cardType == "MoneyBack")
    {
        cardDetails = new MoneyBack();
    }
    else if (cardType == "Titanium")
    {
        cardDetails = new Titanium();
    }
    else if (cardType == "Platinum")
    {
        cardDetails = new Platinum();
    }

    return cardDetails;
}
```

Now, the client needs to get the object through CreditCardFactory. For example, if the client wants to create an instance of a Platinum card, he/she needs to do something like the below. As you can see, he/she needs to pass the Credit card type to the GetCreditcard method of the CreditCardFactory class. Now, the GetCreditcard() method will create a Platinum class instance and return that instance to the client.

```
CreditCard cardDetails = CreditCardFactory.GetCreditCard("Platinum");
cardDetails.GetCardType();
cardDetails.GetCreditLimit();
cardDetails.GetAnnualCharge();
```

Step 4: Creating Factory Class

Create a class file named **CreditCardfactory.cs** and copy and paste the following. This class contains the logic to create and initialize the appropriate object and returns that object based on some condition. As you can see, this class contains one static method. That static method takes one string parameter, and based on the parameter value, it will create and return the appropriate product instance (MoneyBack, Titanium, and Platinum) to the client.

```
namespace FactoryDesignPattern
{
    public class CreditCardFactory
    {
        public static CreditCard GetCreditCard(string cardType)
        {
            CreditCard cardDetails = null;
            if (cardType == "MoneyBack")
            {
                cardDetails = new MoneyBack();
            }
            else if (cardType == "Titanium")
            {
```

```

        cardDetails = new Titanium();
    }
    else if (cardType == "Platinum")
    {
        cardDetails = new Platinum();
    }
    return cardDetails;
}
}
}

```

Step 5: Using Factory Class in Client Code to Get the Product Instance

Now, we need to modify the client code. So, modify the Main method of the Program class as follows. As you can see, we are not creating the Product instance using the IF-ELSE Condition; instead, we are calling the Static GetCreditCard method of the CreditCardFactory class by passing the Credit Card Type, which instance we need to create. Once the GetCreditCard method returns the appropriate product instance, we consume the methods. Here, we are storing the object in the super interface GetCreditCard.

```

using System;
namespace FactoryDesignPattern
{
    class Program
    {
        static void Main(string[] args)
        {
            CreditCard cardDetails =
            CreditCardFactory.GetCreditCard("Platinum");

            if (cardDetails != null)
            {
                Console.WriteLine("CardType : " +
                cardDetails.GetCardType());
                Console.WriteLine("CreditLimit : " +
                cardDetails.GetCreditLimit());
                Console.WriteLine("AnnualCharge :" +
                cardDetails.GetAnnualCharge());
            }
            else
            {
                Console.WriteLine("Invalid Card Type");
            }
            Console.ReadLine();
        }
    }
}

```

Output:

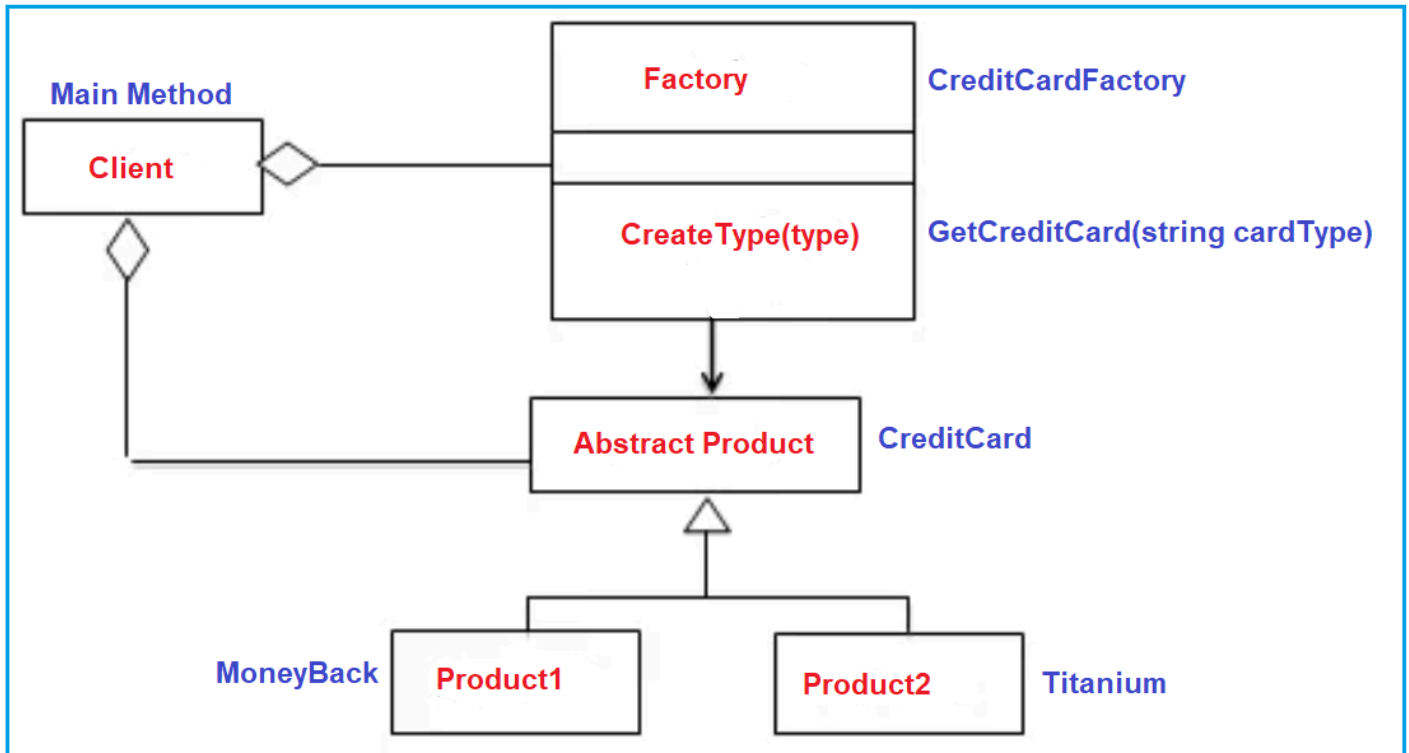
```

CardType : Platinum Plus
CreditLimit : 35000
AnnualCharge :2000

```

Factory Design Pattern UML (Class) Diagram:

So, once we understand how to implement the Factory Design Pattern in C#, let us try to understand the UML (Unified Modeling Language) or Class diagram of the Factory Design Pattern. The Unified Modeling Language is a general-purpose, developmental modeling language in software engineering that is intended to provide a standard way to visualize the design of a system. For a better understanding, please look at the following diagram, which shows the different components of the Factory Design Pattern. Here, I am comparing the Factory Design Pattern UML diagram with our CreditCard example so you can easily understand the concept.



Here's a breakdown of the typical components in a Factory Design Pattern UML diagram:

- **Product Interface / Abstract Product:** This represents the interface or abstract class defining the methods that concrete products must implement. In our example, it is the CreditCard Interface.
- **Concrete Products:** These are the classes that implement the Product interface or extend the abstract class. They are the actual objects that the factory will create. In our example, it is the MoneyBack, Platinum, and Titanium classes.
- **Creator or Factory:** This class provides the factory method to return an instance of a specific Concrete Product. In our example, it is the CreditCardFactory class.
- **Client:** The class that consumes the actual product objects. In our example, it is the Program class's Main method.

Real-Life Example of Factory Pattern:

From Lehman's point of view, we can say that a factory is a place where products are created. In other words, we can say that it is a centralized place for creating products. Later, based on the order received, the factory delivers the appropriate product. For example, a car factory can produce different types of cars. If you order a car based on your requirements or specifications, the factory will create the appropriate car and then deliver that car to you.

The same thing happens in the Factory Design Pattern. A factory (i.e., a class) creates and delivers products (i.e., objects) based on the incoming parameters.

Problems of Simple Factory Pattern in C#

1. If we need to add a new product (e.g., a new credit card), we need to add a new if-else condition in the GetCreditCard method of the CreditCardFactory class. This violates the open/closed design principle.

2. We also have a tight coupling between the Factory (CreditCardFactory) and Product classes (MoneyBack, Titanium, and Platinum). We can overcome these problems using the [Factory Method Design Pattern](#), which we will discuss in our upcoming articles.

When to use the Factory Design Pattern in C# Real-Time Applications?

The Factory Design Pattern is useful in C# real-time applications where the system needs to manage, create, and manipulate multiple objects with a common interface but with different underlying implementations. The following are some specific scenarios where the Factory Design Pattern can be beneficial in real-time applications:

- **Complex Creation Logic:** When the creation of an object is complex and requires more than simple instantiation. The Factory pattern can encapsulate this complexity.
- **Similar Object Families:** When dealing with families of related or dependent objects designed to be used together, you need a way to create these objects so that they can easily be swapped with other families without modifying the client code.
- **Subclass Selection:** When you need to support the dynamic instantiation of one or more derived classes from a common base class or interface based on the input or configuration data provided.
- **System Scalability:** As real-time applications grow, new object types can be added with minimal changes to the existing application code. Factories can handle the creation logic for these new types without altering existing code, enhancing the scalability of the application.
- **Conditional Object Creation:** When the instantiation of an object depends on certain external conditions or configuration settings, and you want to abstract these conditions from the main application logic.
- **Centralized Object Management:** When you need to centralize the control of object creation to ensure consistency in behavior or configuration among the instances that are produced.

In the next article, I will discuss the [Real-time Examples of the Factory Design Pattern in C#](#). Here, in this article, I try to explain the **Factory Design Pattern in C#** with Examples. I hope this article will help you with your needs. I would like to have your feedback. Please post your feedback, questions, or comments about this article.